

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.Doi Number

Development and Experimental Validation of a Python-Based Joint-Level Control Bridge for a Dynamixel-Based Hexapod Robot (June 2026)

Wong Zhen Hao¹, Tiong Yew Tang¹, Chian C. Ho², Muaadh Mukred³, Mohammad Ahmed Alomari⁴, and Warusia Yassin⁵

¹Research Centre for Human-Machine Collaboration (HUMAC), Department of Smart Computing and Cyber Resilience, Faculty of Engineering and Technology, Sunway University, Bandar Sunway, Selangor, Malaysia

²Department of Electrical Engineering, National Yunlin University of Science and Technology, Yunlin County, Taiwan

³Department of Business Analytics, Sunway Business School, Sunway University, Bandar Sunway, Selangor 47500, Malaysia

⁴Department of Cybersecurity, Faculty of Artificial Intelligence and Cyber Security (FAIX), Universiti Teknikal Malaysia Melaka (UTeM), Melaka, Malaysia

⁵Department of Cyber Security, Faculty of Artificial Intelligence and Cyber Security (FAIX), Universiti Teknikal Malaysia Melaka (UTeM), 76100 Melaka, Malaysia

Corresponding author: Wong Zhen Hao (e-mail: wzhenhao06@gmail.com).

This work was conducted as part of the broader Autonomous Oil Palm Harvesting Robot project, with the proposed system serving as the locomotion-control subsystem of the hexapod platform.

ABSTRACT This paper presents the development and experimental validation of a Python-based joint-level control bridge for a Dynamixel-based hexapod robot as a locomotion-control subsystem of a broader Autonomous Oil Palm Harvesting Robot project. The original platform relied on a legacy motion-programming workflow, in which movements were executed through predefined sequences created using existing robotic software. Although this approach is suitable for basic remote-control demonstrations, it limits direct actuator-level experimentation, web-based control, controller-client communication, inverse kinematics, and future learning-based research. The proposed system introduces a modular Python control layer that maps Dynamixel actuator identifiers to physical hexapod joints, stores both code-balanced reference poses and calibrated physical ready-pose values, sends controlled raw-position commands, and reads actuator feedback such as position, voltage, temperature, load, and movement status. A flat-pose actuator-readback calibration method was added to reduce the mismatch between software-defined joint targets and the physically balanced stance of the robot. Additionally, the system supports terminal commands, a browser-based control interface, game-controller input, fixed joint-space gait execution, and preliminary inverse kinematics and Bézier trajectory motion testing. Experimental testing, including a preliminary five-cycle comparison of fixed gait, IK Motion, and IK Bézier Motion, shows that reliable hexapod locomotion depends not only on software commands, but also on voltage stability, actuator temperature, mechanical friction, leg balance, ground contact, wiring reliability, and motor condition. The developed bridge provides a practical foundation for safer movement testing and future implementation of onboard computing, inverse kinematics motions refinement, robotics compatibility and reinforcement learning experiments. This research showed that it is necessary to have a calibrated low-level control bridge prior to deploying advanced autonomous locomotion methods to real hexapod robots.

INDEX TERMS Dynamixel actuator, hexapod robot, joint-level control, Python control bridge, actuator calibration, actuator diagnostics, gait control, inverse kinematics, Raspberry Pi, web-based teleoperation.

I. INTRODUCTION

Hexapod robots are six-legged robotic platforms that are commonly used to study legged locomotion for its stability, gait control experiments, actuator coordination and robot control architecture. Whereas, compared to wheeled robots, a hexapod robot can maintain body support while lifting selected legs during movements, making it especially useful for experiments such as walking gait, load distribution, body posture and terrain adaptation abilities for natural environments. However, controlling a hexapod robot is more complex than controlling a simple mobile robot because each leg normally contains multiple joints that must move in a coordinated and balanced manner. In a physical robot, movement performance is also affected by mechanical friction, wiring reliability, supply voltage, motor temperature, foot contact, body balance, and load distribution.

The hexapod platform used in this work is based on ROBOTIS Dynamixel AX-series smart actuators, including AX-12A and AX-18A motors. The original system relied on a CM-530 controller and RoboPlus Motion workflow, where the movement is commonly created and executed using predefined motion pages. This workflow is useful for remote-control demonstrations because fixed motion sequences can be prepared and replayed. However, it becomes limiting when the objective is to develop a flexible research platform that can support direct joint-level control, web-based teleoperation, controller input, actuator diagnostics, inverse kinematics, onboard computing and reinforcement learning experiments.

To address these limitations, this work develops a Python-based joint-level control bridge for the Dynamixel-based hexapod robot. The proposed bridge separates high-level user commands from low-level actuator execution. User commands can be sent through a terminal interface, a web-based interface, or an external controller-client script. The Python control layer then routes the commands, applies the motor-to-joint mapping, uses calibrated ready-pose values, generates movement targets and sends controlled raw-position commands to the Dynamixel actuator bus. The same bridge also reads actuator feedback values such as present position, voltage, temperature, load and movement status, allowing the system to support both movement execution and hardware observation.

The development process showed that reliable hexapod movement cannot be achieved by software commands alone. During physical testing, the same movement command could produce different results depending on battery voltage, actuator temperature, mechanical resistance, leg balance, ground contact and the condition of individual motors. Therefore, the control bridge was designed not only as a movement interface but also as a

hardware validation and diagnostic layer. This is important because future systems such as inverse kinematics, robotics middleware and reinforcement learning should not directly send unsafe commands to the physical motors without passing through calibration, mapping and safety-checking layers.

The main contributions of this paper are as follows. First, a Python-based joint-level control bridge was developed to reduce dependence on fixed motion-page execution and enable programmable actuator-level control. Second, the system supports terminal control, web-based control, controller-client communication, ready-pose execution, walking, strafing, turning, body-height adjustment, and diagnostic monitoring. Third, a calibrated motor-to-joint mapping and flat-pose actuator-readback method were created to support repeatable ready-pose execution by compensating for horn alignment differences, mirrored leg installation, mechanical tolerances, and specific motor offsets. Fourth, practical experimentation of the hardware are observed and were documented including the effects of voltage drop, actuator temperature, load variation, mechanical friction, leg balance and wiring reliability on movement consistency. Finally, the developed bridge provides a foundation for future onboard Raspberry Pi control testing, inverse kinematics movement refinements, robotics middleware compatibility, and reinforcement learning-based locomotion research.

The remainder of this paper is organized as follows. Beginning with, Section II which reviews related work on hexapod platform including Dynamixel actuator controls, fixed gait movement and inverse kinematics motion experiments, as well as modular robot control software. Section III describes the proposed system architecture and hardware setup. Section IV presents the implementation, testing process, diagnostic tools, movement results, and practical observations. Section V concludes the paper and outlines future development directions.

II. RELATED WORK

A. Hexapod Robot Locomotion

Hexapod robots are widely studied in the field of legged locomotion because of their six-legged structure which allows for multiple selected legs to remain in contact with the ground while other legs are lifted during movement known as gait controls and motions. This makes them suitable for experiments involving static stability, gait coordination and legged locomotion. Additionally, A common movement and approach is the tripod gait, where three legs support the body while the other three legs are lifted and move forward. This gait is useful because it provides a balance between movement speed and body support. However, real hexapod movement depends not only on gait patterns but also on the robot's

mechanical structure, actuator placement, leg geometry, motor response and ground contact conditions [2].

In a practical hardware testing environment, the same gait command may produce different movement results depending on voltage level, mechanical friction, joint alignment, actuator condition and body balance. Therefore, a hexapod control system should not only generate movement commands but also provide calibration, feedback monitoring and a safe command execution layer. This is especially important when the robot is reused, modified, or developed from an older hardware platform.

B. Dynamixel Actuator Control

Dynamixel smart actuators are commonly used in educational and research robots because each actuator integrates a motor, reduction gear, controller, driver, sensor feedback and serial communication interface. The AX-series actuators used in this project allows the control system to send target position commands while also reading feedback values such as present position, voltage, temperature, load and moving status [4]. These feedback values are especially useful for observing not just the physical but also software conditions of the robot during testing and deployments.

For a multi-joint hexapod robot, actuator identification and motor-to-joint mapping are essential for both testing, debugging and deployment phase. If the motor identifiers are not correctly mapped to physical joints, a command intended for one joint may move another joint, causing unstable posture, incorrect movement, and mechanical strain, which hexapod robots are especially prone to damages caused by mis-mapped joint because it contains 3 categories of joints Coxa, Femur and Tibia. Therefore, the control bridge developed in this work uses a calibrated mapping between Dynamixel actuator identifiers and the physical hip, femur, and tibia joints of the six legs. Additionally, packet-based communication with the actuators is supported through the Dynamixel software interface and development tools [5].

C. Inverse Kinematics for Legged Robots

Inverse kinematics is an important technique used in legged robot movement controls because it converts a desired foot position into the joint angles required to reach that position through trigonometry such as law of cosines and sines. Instead of manually tuning raw motor offsets for every movement which lowers the movement speed, efficiency and stability, inverse kinematics allows the robot to be controlled using foot-space targets. This is useful for future movement refinement because walking, strafing, turning and body posture control can be described using target positions rather than only individual joint commands [1].

However, inverse kinematics alone does not guarantee reliable physical movement. Nevertheless, even if the mathematical joint angles and horn alignment are correct, the real robot still requires servo calibration, direction correction, safe joint limits, actuator feedback monitoring, and physical validation. In this project, inverse kinematics and Bézier trajectory testing were implemented as advanced movement modes, while the physically tested joint-space gait remains available as a stable fallback and comparison baseline.

D. Modular Robot Software and Middleware

Modern robot systems often separate user inputs, command routing, movement planning, actuator execution and diagnostic monitoring into different layers. This separation improves maintainability and allows future systems such as web interfaces, controller clients, robotics middleware or learning-based controllers to interact with the robot through a structured command path [3]. However, with the approach of distributing and separation of layers across multiple programs or services can introduce increases communication latency, synchronization issues, duplicated configuration, additional failure points, and complicated startup and debugging procedures. In addition to that, it may also cause conflicts when multiple processes attempt to access the same serial actuator bus at the same time.

In this project, custom Python scripts perform middleware-like functions before full autonomous robotics integration. The Python control bridge first receives commands from the terminal, web interface, or controller client, then processes the commands through the movement layer by applying motor mapping and calibration values before sending controlled position targets to the Dynamixel actuator bus. This modular structure helps prevent high-level commands from directly controlling raw actuator values without passing through calibration and safety logic.

III. SYSTEM ARCHITECTURE AND HARDWARE SETUP

A. OVERALL SYSTEM ARCHITECTURE

The proposed system is designed as a joint-level control bridge between high-level user commands and low-level Dynamixel actuator execution. The original robot platform was built around a CM-530 controller and a Dynamixel actuator bus. Instead of replacing the entire hardware platform, this work retains the existing Dynamixel-based structure and introduces a Python control layer that can communicate with the actuator system through the available serial or USB-to-Dynamixel communication path.

The overall architecture consists of three main layers: the user command layer, the Python control bridge layer and the actuator execution layer. The user command layer includes terminal commands, a web-based interface and a controller-

client script. These interfaces allow the user to send high-level commands such as ready pose, forward movement, backward movement, strafing, turning, stopping, body-height adjustment, push-up motion, speed tuning and diagnostic checks. The Python control bridge receives these commands, routes them to the correct movement or diagnostic function, applies motor mapping and calibration values, and sends the final raw-position targets to the Dynamixel actuators.

The actuator execution layer consists of 18 ROBOTIS Dynamixel AX-series smart actuators arranged across six legs. Each leg contains three joints: hip or coxa, femur, and tibia. The Dynamixel actuators are connected through a shared communication bus, allowing the controller to send commands to individual actuator identifiers while also reading feedback values from the motors.

TABLE I
CONTROL BRIDGE CALIBRATION DATA USED FOR HEXAPOD MOVEMENT

Calibration item	Purpose
Motor identifier mapping	Links each Dynamixel actuator ID to a physical leg and joint
Code-balanced and calibrated ready-pose values	Stores both the common software reference pose and the motor-specific physical ready-pose offsets
Joint direction sign	Corrects mirrored or reversed servo installation directions
Movement offset	Converts logical movement commands into raw actuator targets
Practical joint limit	Restricts movement based on tested physical hardware range
Diagnostic feedback	Records position, voltage, temperature, load and moving state

B. HARDWARE CONFIGURATION

The hexapod robot uses AX-series Dynamixel actuators, including AX-12A and AX-18A motors. The AX-18A actuators are used in joints where higher load is expected, while AX-12A actuators are used in lower-load positions. The controller and actuator system are connected through the existing CM-530 and Dynamixel bus structure. A Universal Serial Bus (USB)-to-Dynamixel or serial communication adapter is used to allow the Python control script to send actuator commands from a laptop or onboard computing device.

The robot is powered by a 3-cell lithium-polymer battery, which provides the main voltage supply for the robot. Because the Raspberry Pi requires a lower regulated voltage, a step-down voltage regulator is used to reduce battery voltage to a safe level for onboard computing. The intended onboard power structure separates the actuator power path from the Raspberry Pi power path. The actuator branch supplies the CM-530 and Dynamixel motor system, while the computing branch supplies the Raspberry Pi through the step-down regulator.

The Raspberry Pi deployment was validated as an onboard Python server path for the control bridge. The Raspberry Pi can host the web-based control interface and communicate with the CM-530/Dynamixel actuator system through the selected USB or serial communication path, allowing external devices on the same local network to send movement commands through a browser or controller-client script. Additionally, this approach of the control bridge allows access not limited to only computer but also other devices under the same network for development setup and can be moved toward future onboard operations. However, longer-duration untethered testing, mechanical mounting refinement, and extended battery-powered validation remain future work.

C. MOTOR-TO-JOINT MAPPING

Accurate motor-to-joint mapping is required before any reliable movement can be developed. Since the robot has 18 actuators, each Dynamixel identifier must be linked to a specific physical joint. Afterwards, the control bridge stores this mapping so that movement functions can refer to meaningful joint names instead of only numerical motor identifiers. Through this approach it also improves debugging because abnormal actuator behavior can be traced to a specific leg and joint.

The motor mapping used in this project connects each actuator comprised to one of the six legs: front left, front right, middle left, middle right, rear left and rear right. While each leg contains 3 categories of joints, a hip or coxa joint, a femur joint, and a tibia joint. This mapping allows the controller to generate coordinated gait behavior, apply direction correction, perform pose calibration and display actuator feedback in a readable form.

D. POWER AND COMMUNICATION FLOW

The power and communication design plays an important role in this project because the hexapod requires both stable power and reliable command transmission. Furthermore, the motor system requires sufficient current to move multiple actuators, while the Raspberry Pi requires a stable regulated voltage for computing and network communication. Conversely, a weak or unstable supply can cause movement inconsistency, actuator resets, communication errors, unreliable robot behavior or cause potential damage to computer components.

In the proposed flow, external user devices such as a laptop, web browser, or game controller communicate with the Python server through a local network. The Python server processes the command and sends the corresponding actuator targets through the Dynamixel communication path. This structure allows the robot to be controlled through different input methods while keeping the low-level actuator execution inside the Python control bridge. As a result, future implementations and improvements such as inverse kinematics, robotics middleware or reinforcement learning models can send high-level commands to the bridge without directly controlling raw motor values.

IV. IMPLEMENTATION, TESTING, AND RESULTS

A. CONTROL BRIDGE IMPLEMENTATION

The implemented control bridge developed in this research provides a reliable command layer for users to control robot movement execution safely. Additionally, the system accepts commands from three main interfaces: a terminal interface, a web-based interface, and a controller-client script. Firstly, The terminal interface was used during development because it allows for direct command entry, fast and comprehensive debugging. Secondly, the web-based interface was developed to make the robot easier to operate during testing and demonstration. Finally, the controller-client script which allows a game controller to send high-level movement commands to the Python server through a local network connection.

The implemented command set includes ready pose, stop, forward movement, backward movement, strafe left, strafe right, turn left, turn right, body-height adjustment, push-up motion, health checking, movement mode switching and speed control. These commands are not sent directly to the actuators, but instead first passed into Python control bridge to be processed. Which the script then applies 3 different processes, starting with applying the correct movement functions, calibration offset values and then finally the actuator target positions before sending the final raw-position values to the Dynamixel bus for safe execution.

To support reproducibility, the implementation files, dependency list, setup instructions, calibration notes, and movement-test workflow are provided in a public source-code repository [8] and Code Ocean capsule [9]. The repository allows reviewers to reproduce the software workflow by installing the required Python packages, selecting the correct CM-530/Dynamixel serial communication path, running the control bridge, placing the robot in the calibrated ready pose, and executing the fixed gait, IK Motion, and IK Bézier Motion test modes. Since the system depends on physical hardware, exact walking distance, actuator temperature, and timing may

vary with battery voltage, floor contact, actuator condition, mechanical alignment, and manual stopping latency.

A variety of functions are included in the control interfaces for the flexibility of control such as speed control. Considering that, physical gait performance depends strongly on timing. For instance, if the gait sequence runs too quickly, the actuators may not reach their target positions before the next phase begins, causing dragging and incomplete movement cycle, weak movement or unstable posture. Inversely, if the gait is too slow, which can overload the motors quicker due to holding a position for too long in imbalance surfaces causing the robot to become less reliable for demonstrations for a long period of time. Therefore, speed and delay parameters were adjusted and optimized during testing to balance movement smoothness and speed, body stability and actuator load.

B. CALIBRATION AND DIAGNOSTIC TOOLS

Calibration was required before full-body movement could be tested safely. The first step of the calibration stage involves identifying the active Dynamixel actuators and mapping each motor identifier to its physical leg and joint for easier human identification and diagnosis. The second stage involves a flat-pose actuator-readback calibration method, where the robot is first commanded into a common code-balanced reference pose using clean raw-position values for each category of joints such as hip/coxa, femur, and tibia joints. Afterwards, the legs are then physically adjusted into a flat, spread out pose and more mechanically balanced stance, after which the Dynamixel present-position feedback is recorded from each actuator. The difference between the commanded reference value and the physically balanced readback value is treated as a motor-specific calibration offset. These offsets are then stored in the control bridge and used to generate a calibrated physical ready pose.

This calibration method is developed to help compensate for horn alignments offsets and differences to achieve a more consistent result during gait movement execution testing. As a result, the ready pose is no longer treated only as a software defined set raw values but as a physically validated reference posture derived from actuator feedback and real physical robot alignment.

Direction correction also plays a major role in producing consistent movement and results during testing. Since, the mechanical orientation of motors differs between legs and mechanical joints offset differences. Therefore, a positive raw-position change may produce opposite physical movement depending on the joint and side of the robot. For this reason, the controller applies joint-specific direction signs and leg-specific movement signs before sending actuator targets.

The diagnostic tools were used to reading actuator feedback values, including present position, voltage, temperature, load,

moving state, and warning condition, where it is based on Dynamixel diagnostic and troubleshooting guidance [6]. Furthermore, these functions allowed the robot to be checked before and after movement testing. Voltage readings were used to observe power stability, temperature readings were used to detect actuator heating, and load readings were used as approximate indicators of stress or resistance. The load value was not treated as an exact torque measurement because it can be affected by ground contact, joint angle, mechanical friction, actuator condition, and body balance.

Joint-limit probing was also used as part of the testing process. Although the Dynamixel actuator has a theoretical movement range, the assembled robot has practical mechanical limits caused by bracket geometry, horn alignment, cable routing, body collision, and mechanical resistance. Therefore, safe movement boundaries must be discovered through physical testing instead of assuming that the full actuator range is usable.

C. MOVEMENT AND GAIT TESTING

Movement testing was developed progressively from individual joint control to full-body gait execution testing relying on visual observation. Initial tests focused on motor scanning, single-joint movement, leg lifting, and ready-pose stability. After the robot could stand reliably, walking behavior was tested using fixed joint-space movement offsets. The fixed gait mode became the most stable baseline because it was tuned directly on the physical robot compared to directly programming movement through code-based alone, making it a suitable candidate to be used as a reference for inverse kinematics movement experiments.

The implemented movement behaviors include forward walking, backward walking, strafing, turning, body-height adjustment, push-up motion, and stop behavior. Tripod-style gait logic was used for walking, where one group of legs moves while the other group supports the robot body. Additionally, the gait coordination and movement were refined through lift-height tuning, support-push adjustment, timing changes, and semi-overlapping movement phases.

Throughout the development and testing it has showed that movement quality highly depended on both software timing and physical actuator response. In early tests, some movement sequences appeared correct in code but produced weak or incomplete movement because the motors did not have enough time to complete its current phase for proper movement execution. In other cases, it was found that insufficient lift caused the feet to drag against the ground, while excessive lift increased actuator load creating a loop of errors from quick overheating or overloading of specific motors. As a result, these observations showed that practical gait tuning requires balancing clearance, speed, stability, voltage behaviour, and motor load rather than optimizing one factor alone.

D. WEB-BASED AND CONTROLLER-BASED TESTING

The web-based interface successfully allowed movement commands to be sent from a browser. The primary reason of web-based control is to reduce the need to repeatedly type terminal commands and make the robot easier to operate during physical testing and demonstrations. Additionally, web interface control can also be used as a validation layer for commands routing. It helps to confirm that the movement commands can be sent through the network interface instead of limited to only terminal inputs.

The controller-client script also confirmed that external controller input could be separated from low-level motor execution. In this setup, the game controller sends high-level command requests to the Python server, while the server remains responsible for movement generation, calibration, safety logic, and actuator communication. This separation is important because future input sources, such as robotics middleware or reinforcement learning policies, should not directly command raw actuator values without passing through the control bridge.

E. INVERSE KINEMATICS AND Bézier MOTION TESTING

Inverse kinematics was experimented and implemented into the controls as an advanced movement mode to support foot-space control. Instead of relying only on fixed raw-position offsets, the inverse kinematics layer converts target foot displacement into hip, femur, and tibia movement commands based on the measured leg segment lengths. This allows the robot to be controlled using movement targets in foot space, which is useful for future gait refinement and learning-based control.

Bézier trajectory motion was also implemented into the control script as part of the experimental feature that can be added into other motion modes such as inverse kinematic to generate smoother leg swing paths. This approach allows the foot to follow a curved trajectory instead of moving through abrupt position changes which demonstrates benefits to motors. However, physical testing showed that smoother mathematical trajectories do not automatically produce better real-world walking. Larger lift arcs and more aggressive swing paths can increase actuator load, contribute to voltage sag, or reduce movement consistency. Therefore, the inverse kinematics and Bézier modes are treated as active experimental features, while the fixed gait mode remains the stable baseline for demonstration.

F. PRACTICAL RESULTS AND DISCUSSION

The implementation and results shows that the developed Python-based control bridge has successfully transformed the robot from a predefined motion-page control platform into a programmable joint-level experimental platform for future research and experiments such as implementation of reinforcement learning. The system can receive commands from terminal, web-based, and controller-client interfaces;

route commands through the Python bridge; apply calibration data; generate raw actuator targets; and read feedback from the Dynamixel motors.

To provide preliminary quantitative validation, a five-cycle walking comparison was performed using the same speed setting, starting line, floor surface, and front-body measurement reference. Before each test the robot was returned to the calibrated ready pose, and actuator temperature was recorded before and after each five-cycle run. The timing uncertainty was approximately ± 0.8 s because of manual command and stopping latency.

Table II Preliminary five-cycle movement comparison at speed setting 50

Mode	Time (s)	Distance (cm)	Speed (cm/s)	Max Temp. (°C)
Fixed gait	46.06	57.0	1.24	33→43
IK Motion	23.80	63.0	2.65	40→43
IK Bézier	23.40	62.5	2.67	36→44

The fixed gait fallback is developed and programmed phase by phase from lifting to moving forward, and shown to produce the steadiest movement but required more time to complete the five-cycle test because of its conservative gait timing. IK Motion and IK Bézier Motion completed the same five-cycle test faster and produced slightly longer forward displacement under the tested speed setting. However, these values are treated as preliminary physical validation rather than a complete gait-efficiency benchmark because battery condition, floor contact, actuator load, starting temperature, and manual timing latency can affect the measured results. As a result, the comparison confirms that the developed bridge supports both stable fallback motion and more efficient experimental movement modes while keeping actuator temperature and movement behavior observable.

Throughout development and testing it has been confirmed that a reliable hexapod movement requires more than sending target positions to motors. For instance, voltage level, actuator temperature, load limitations, mechanical frictions, wiring reliability, leg balance and ground contact which can all affect the consistency of movement or results. A gait that appeared stable under one battery level or stance condition could become weaker or less consistent under another condition. This confirmed the need for diagnostic feedback and calibration tools as part of the control system.

The results also showed that the control bridge can provide an additional safety check before more advanced control methods, such as inverse kinematics, robotics middleware, or reinforcement learning are used. These implementations and methods may produce high-level movement commands, but

the commands still need to go through the calibrated control bridge before being sent to robot motors. As a result, from checking and adjusting the commands first, the system can lower the chance of sudden unsafe movement, poor body balance, excessive motor strain, or the robot parts colliding with each other.

The onboard Raspberry Pi server path was validated as part of the control bridge architecture, confirming that the system can operate on other devices and not limited to a laptop-only development setup. However, longer untethered operation, improvements to the mechanical mounting, and further battery-powered testing are still ongoing. Additionally, reinforcement learning and full robotics middleware integration are also planned as future development stages rather than being included as completed results in this project. Therefore, the main contribution of this project is the development of a practical and calibrated control bridge with diagnostic features, as well as providing a safer and more reliable foundation for future research and experiments on hexapod locomotion.

V. CONCLUSION

This paper presents the development and experimental testing of a Python-based joint-level control bridge for a Dynamixel hexapod robot. The purpose of the system that was developed in this research is to reduce dependence on predefined motion-page execution for movement and controls, and to provide a more flexible control structure for actuator-level testing, web-based teleoperations, controller-client communication, gait development, diagnostic monitoring, and future advanced locomotion research.

The developed bridge separates high-level user input from low-level actuator execution. Commands from the terminal, web interface, and controller-client script are routed through the Python control layer before being converted into calibrated Dynamixel raw-position targets. Additionally, the system also supports ready-pose calibration, motor-to-joint mapping, direction correction, speed tuning, diagnostic feedback reading, and safe movement testing. These functions developed and implemented into the script allowed the robot to perform fixed joint-space gait movements, including forward movement, backward movement, strafing left and right, turning, body-height adjustment, and push-up motion, while also supporting preliminary inverse kinematics and Bézier trajectory experimental motions modes.

The experimental results showed that stable hexapod movement depends not only on the control software, but also on the condition and performance of the physical hardware. For instance, voltage stability, actuator temperature, mechanical friction, motor load, leg balance, ground contact,

wiring reliability, and actuator condition all which can affect movement consistency and results. Therefore, with these factors considered, a five-cycle comparison test was carried out to evaluate the different movement modes. Subsequently, the results showed that the IK-based movement mode completed the test faster, as well as travelled further than the fixed gait mode. However, the fixed gait movement mode remained useful as a more stable and reliable baseline for comparison.

Future work will focus on longer wireless and battery-powered testing, as well as improving the mounting of the onboard computer by distributing the weight more evenly between the front and rear of the hexapod. Besides that, further calibration and adjustment will also be carried out to improve the inverse kinematics and Bézier-based movement. Beyond the existing features, future work will include verifying joint safety limits, improving compatibility with robotics middleware, and preparing the system for reinforcement learning for autonomous and adaptive locomotion. Overall, the completed bridge provides a practical foundation for transforming a legacy motion-page-based hexapod into a programmable and extendable robotics research platform.

APPENDIX A

SUPPLEMENTARY IMPLEMENTATION EVIDENCE

A. HARDWARE AND COMMUNICATION ARCHITECTURE

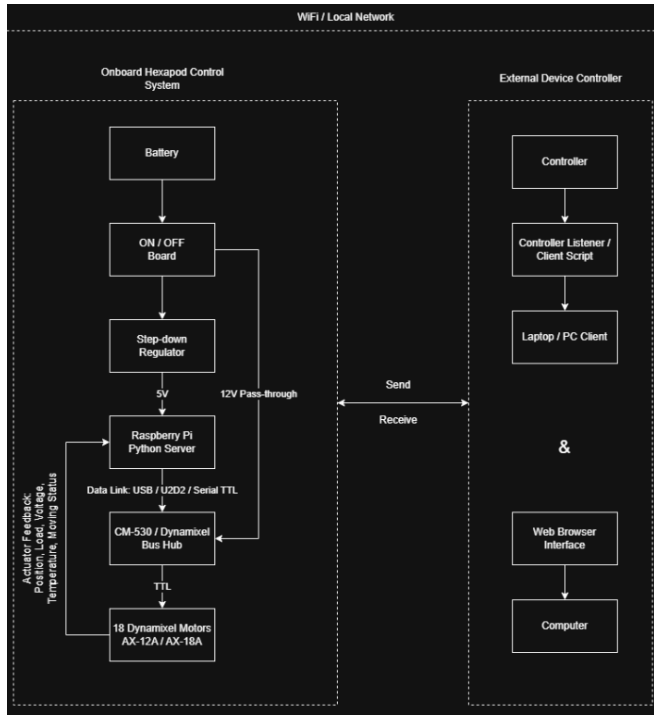


FIGURE A1. Proposed onboard and external control architecture of the Python-based hexapod control system.

Fig. A1 presents the proposed onboard and external control architecture of the hexapod platform. The onboard system includes the battery, step-down regulator, Raspberry Pi Python server, CM-530/Dynamixel communication path, and 18 Dynamixel actuators. External commands are transmitted through a local network from either a controller-client script or a web-browser interface, while actuator feedback is returned to the control system for monitoring and diagnostic purposes.

B. WEB-BASED CONTROL AND DIAGNOSTIC INTERFACE

The web-based interface provides movement control, stance-height adjustment, speed and gait tuning, movement-mode selection, health checking, terminal feedback, and live robot status monitoring. As shown in Fig. A2, the interface combines user commands and diagnostic information within a single browser-based control panel, allowing commands to be routed through the Python control bridge before reaching the Dynamixel actuator system.

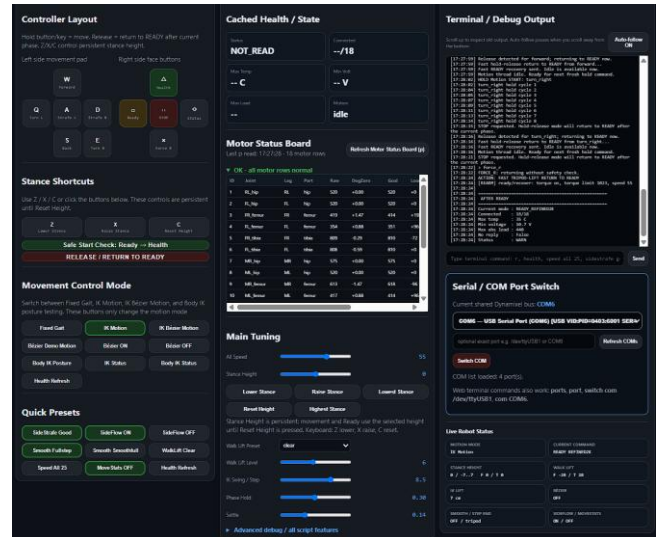


FIGURE A2. Web-based movement control interface, tuning controls, terminal output, and live diagnostic status panel.

C. MOVEMENT AND DIAGNOSTIC EVIDENCE

Testing evidence included ready-pose validation, actuator status monitoring, forward and backward movement, strafing, turning, body-height adjustment, fixed gait testing, inverse kinematics motion, and Bézier trajectory experiments. Diagnostic outputs were used to observe present position, voltage, temperature, load, moving state, and warning conditions before and after movement tests.



(a) calibrated ready-pose



(b) physical gait movement

SControlX2 command [h help]: p

```
=====
MOTOR STATUS / FULL DIAGNOSTICS
=====
```

ID	Joint	Leg Part	Raw	DegZero	Zero	Goal	Load	Volt	Temp	Warnings
1	RL_hip	RL hip	385	-21.99	460	460	+0	12.0	29	OK
2	FL_hip	FL hip	749	+0.59	747	747	+0	12.1	31	OK
3	FR_femur	FR femur	726	+92.38	411	411	+0	12.0	30	OK
4	FL_femur	FL femur	650	+83.28	366	366	+0	12.0	31	OK
5	FR_tibia	FR tibia	454	-100.88	798	798	+0	12.3	30	OK
6	FL_tibia	FL tibia	433	-106.45	796	796	+0	12.0	29	OK
7	MR_hip	MR hip	539	-20.23	608	608	+0	12.0	29	OK
8	ML_hip	ML hip	847	+27.57	753	753	+0	11.8	31	OK
9	MR_femur	MR femur	270	-104.69	627	627	+0	12.1	30	OK
10	ML_femur	ML femur	805	+107.92	437	437	+0	12.0	29	OK
11	MR_tibia	MR tibia	421	+60.12	216	216	+0	12.0	29	OK
12	ML_tibia	ML tibia	421	-107.33	787	787	+0	11.8	30	OK
13	RR_hip	RR hip	486	-26.98	578	578	+0	12.1	29	OK
14	FR_hip	FR hip	654	+23.17	575	575	+0	12.0	29	OK
15	RR_femur	RR femur	252	-114.08	641	641	+8	12.3	29	OK
16	RL_femur	RL femur	752	+99.71	412	412	+0	12.0	29	OK
17	RR_tibia	RR tibia	584	+115.84	189	189	+0	12.1	31	OK
18	RL_tibia	RL tibia	466	-102.93	817	817	+0	11.8	31	OK

(c) actuator diagnostic monitoring.

FIGURE A3. Selected implementation evidence: (a) calibrated ready-pose, (b) physical gait movement, and (c) actuator diagnostic monitoring.

D. SOURCE CODE AND REPRODUCTION WORKFLOW

The source-code repository contains the Python control bridge, web-control files, controller-client interface, calibration configuration, dependency list, and reproduction instructions for the reported movement tests. The recommended reproduction workflow is to install the dependencies, connect the CM-530/Dynamixel communication path, select the correct serial port, run the main control script, enter the calibrated ready pose, set the speed to 50, and repeat the five-cycle comparison using fixed gait, IK Motion, and IK Bézier Motion.

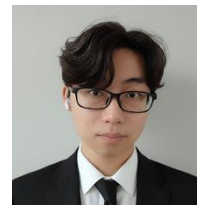
ACKNOWLEDGMENT

The author would like to thank the HUMAC robotics research lab and Sunway University for providing the research environment, hardware platform, and support required for the development and testing of the hexapod robot control system. The author also acknowledges the guidance and feedback received throughout the control-bridge development, actuator calibration, movement testing, diagnostic validation, and documentation process.

OpenAI's ChatGPT was used to assist with manuscript structuring, language refinement, grammar checking, and formatting guidance for the Abstract, Introduction, System Architecture, Implementation, Results and Discussion, Conclusion, and biographical sections [7]. The AI tool was not used to perform the technical implementation, experimental testing, hardware validation, data interpretation, or final research conclusions. The technical implementation, experimental testing, hardware observations, interpretation of results, and final responsibility for the manuscript remain with the author.

REFERENCES

- [1] J. A. Lizarraga, J. A. Garnica, J. Ruiz-Leon, G. Munoz-Gomez, and A. Y. Alanis, "Advances in the kinematics of hexapod robots: An innovative approach to inverse kinematics and omnidirectional movement," *Applied Sciences*, vol. 14, no. 18, Art. no. 8171, 2024, doi: 10.3390/app14188171.
- [2] J. Ma, S. Qiu, J. Yang, H. Zhang, and X. Zhang, "Design, analysis and experiments of hexapod robot with six-link legs for high dynamic locomotion," *Micromachines*, vol. 13, no. 9, Art. no. 1404, 2022, doi: 10.3390/mi13091404.
- [3] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "ROS: An open-source robot operating system," in *Proc. ICRA Workshop Open Source Software*, Kobe, Japan, 2009.
- [4] ROBOTIS, "DYNAMIXEL AX-12A e-Manual," *ROBOTIS e-Manual*. [Online]. Available: <https://emanual.robotis.com/docs/en/dxl/ax/ax-12a/>. Accessed: Jun. 11, 2026.
- [5] ROBOTIS, "DYNAMIXEL SDK," *ROBOTIS e-Manual*. [Online]. Available: https://emanual.robotis.com/docs/en/software/dynamixel/dynamixel_sdk/. Accessed: Jun. 11, 2026.
- [6] ROBOTIS, "DYNAMIXEL FAQ," *ROBOTIS e-Manual*. [Online]. Available: https://emanual.robotis.com/docs/en/popup/faq_dynamixel/. Accessed: Jun. 11, 2026.
- [7] OpenAI, "ChatGPT," OpenAI. [Online]. Available: <https://chat.openai.com/>. Accessed: Jul. 9, 2026.
- [8] W. Z. Hao, "Hexapod Research – Web-Based Joint-Level Control Bridge for a Dynamixel Hexapod," GitHub repository. [Online]. Available: <https://github.com/CookiesWithCream/Hexapod-Research>. Accessed: Jul. 15, 2026.
- [9] W. Z. Hao, "Hexapod Research – Web-Based Joint-Level Control Bridge for a Dynamixel Hexapod," Code Ocean capsule. [Online]. Available: <https://codeocean.com/capsule/2265654/tree>. Accessed: Jul. 20, 2026.



WONG ZHEN HAO is currently pursuing the Diploma in Information Technology at Sunway University, Subang Jaya, Malaysia. He is also a Student Researcher with the HUMAC research laboratory under the School of Engineering and Technology, Sunway University.

His current work focuses on the development of a Dynamixel-based hexapod research platform, including Python-based joint-level control, web-based teleoperation, actuator calibration and diagnostics, inverse kinematics, gait development, Raspberry Pi integration, and preparation for future reinforcement learning experiments. His research interests include robotic locomotion, intelligent control systems, hardware–software integration, quantitative computing, and applied artificial intelligence.